Ders 7 Genişletilmiş Notu: Tek Yönlü, Çift Yönlü ve Dairesel Bağlı Listeler

Ders 7 Genişletilmiş Notu: Tek Yönlü, Çift Yönlü ve Dairesel Bağlı Listeler

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Neden Bağlı Liste?
- 2. Düğüm Yapısı ve Bağlı Listenin Temel Özellikleri
- 3. Tek Yönlü Bağlı Listede Düğüm Oluşturma
- 4. Başa Ekleme
- 5. Sona Ekleme
- 6. Sıralı Listeye Araya Ekleme (Alıştırma)
- 7. Listeyi Ekrana Yazdırma
- 8. Listeden Eleman Silme
- 9. Çift Yönlü Bağlı Liste
- 10. Dairesel Bağlı Liste
- Örnek Sorular
- Dersin Toparlaması

1. Neden Bağlı Liste?

Hoca derse, geçen hafta başlanan bağlı liste (linked list) konusunu hatırlatarak başladı. Diziler, aynı amaç için kullanılan bir blok bilgiyi bellekte ardışık olarak tutmak için uygundur. Fakat dizinin uzunluğu çok değişken olabilir. Örneğin bir sıralama ihtiyacı 50 eleman için de doğabilir, 500 bin eleman için de. Bu durumda elimizde üç seçenek vardır:

- **Statik dizi:** Boyut program içinde sabit yazılır. Ya olabilecek en büyük boyutu vermek ya da programı kullanılacağı ortama göre her seferinde değiştirmek gerekir.
- **Dinamik bellek ayırma (dynamic memory allocation):** Programlama dersinde gösterildiği gibi, eleman sayısı n program çalışmaya başladığında dışarıdan okunur ve o kadarlık yer ayrılır (allocate edilir).
- **Üçüncü durum:** Kaç eleman için işlem yapılacağı, program çalışmaya başladığı anda bile belli değildir. Elemanlar zaman içinde eklenir, işleri bitince çıkarılır.

Hocanın üçüncü durum için örnekleri hareketli yapılardı. Bir mail sunucusunda (mail server) mailler gelir, eklenir, bir kısmı silinir: 10 tane gelir, 2 tanesi silinir. Bir yazıcı kuyruğunda (printer queue) baskı işleri arka arkaya eklenir, işi biten çıkıp gider. Bu verilerin hepsi aynı amaç için kullanılır, tek tek değişkenlerle gösterilemeyecek kadar çoktur, ama sayıları baştan bilinmez. Hoca bu durum için en uygun yapının bağlı liste olduğunu söyledi.

Bağlı listede her eleman geldiği anda yalnız o eleman için yer tutulur. Hocanın örneğinde `head` önce 1000 numaralı adresteki 5'i gösterir. Bir süre sonra 2000 numaralı adreste 9 gelir ve 5'in düğümü 2000'i gösterir. Sonra 2200 numaralı adreste 7 gelir ve 9'un düğümü 2200'ü gösterir. Son düğüm hiçbir yeri göstermez, yani `NULL`'dır.

```
head -> [5 | 2000] -> [9 | 2200] -> [7 | NULL]
         (1000)        (2000)        (2200)
```

Burada alıştığımız şekilde ardışık bir yer ayırma yoktur; örneğin 1000'den 4000'e kadar bir seferde tutulmuş bir blok yoktur. Zamanı geldikçe her eleman için ayrı yer tutulur. Her tutulan elemanı bir başkası gösterdiği için elemanlar kaybedilmeden erişilebilir. Bütünlük, düğümlerin birbirini göstermesiyle sağlanır.

2. Düğüm Yapısı ve Bağlı Listenin Temel Özellikleri

Bağlı listenin elemanları yalnız tek bir değerden oluşmak zorunda değildir. Bir düğüm bir karakter, bir metin ve bir sayı gibi birden fazla bilgiden oluşabilir. Önemli olan, her düğümün kendisinden sonraki düğüme bir bağlantı (link) tutmasıdır. Yani her düğümde iki tür alan vardır: düğümün normal bilgileri (değer, metin vb.) ve kendisinden sonraki düğümün adresi.

Sonraki düğümün adresini tutan alan, aynı düğüm yapısındaki başka bir düğümü göstermelidir. Bu yüzden `next` alanı `struct node *` tipinde tanımlanır. Hoca bu tanımın "özyinelemeli (recursive) bir gösterim" olduğuna dikkat çekti: Yapı, kendi tipinden bir değişkeni gösteren bir pointer içerir.

> **Ders sonrası not:** Hoca bir önceki derste tahtaya `structure` yazdığını fark etti ve düzeltti: C'de anahtar kelime `struct`'tır. Hoca, yaptığı bu tür hataların ders sırasında söylenmesini istedi.

2.1 Sıralı Erişim

Hoca bağlı listenin bir dezavantajını vurguladı: Elemanlara sırayla erişmek zorundayız. Ortadaki bir 9'a doğrudan girip erişmek mümkün değildir. Tasarıma bağlı olarak baştan ya da sondan başlanabilir, ama sırayla ilerlenmek zorundadır.

> **Ek bilgi (derste anlatılmadı):** Dizide i. elemana adres hesabıyla doğrudan erişilir, bu yüzden erişim $O(1)$'dir. Bağlı listede i. düğüme ulaşmak için baştan i bağlantı izlenir; erişim en kötü durumda $O(n)$'dir. Buna karşılık bağlı listede, yeri bilinen bir düğümün yanına ekleme ya da çıkarma, elemanları kaydırmadan birkaç pointer atamasıyla yapılır.

2.2 Tek Yönlü Bağlı Liste ve Temel İşlemler

Geçen hafta görülen tek yönlü bağlı listede (singly linked list) her eleman yalnız bir başka elemanın adresini saklayabilir. İleri yönlü (forward) bağlantılar vardır: Her eleman kendisinden sonrakini, son eleman ise `NULL`'ı gösterir. Bağlı liste üzerindeki temel işlemler şunlardır:

- Yeni eleman ekleme (insert).
- Var olan bir elemanı silme (delete).
- Eleman arama (search).
- Elemanların hepsini ya da istenenleri ekranda gösterme (display).

İlk elemanı gösteren ayrı bir değişken gerekir. Buna genelde `head` denir; `start`, `begin` gibi istenen ad verilebilir. Hoca, listenin sonunu gösteren değişkene de "tail" dendiğini belirtti. Örnek: `head` 1000'i gösterir, 1000'de 9 vardır ve 1200'ü gösterir; 1200'de 7 vardır ve `NULL`'ı gösterir. Bu derste kolaylık için her düğümde yalnız `value` adlı bir değer alanı olduğu varsayıldı.

2.3 Ekleme Türleri ve Ne Zaman Hangisi Kullanılır

Hoca eklemenin üç şekilde olabileceğini söyledi: başa ekleme, sona ekleme ve araya ekleme.

- **Sona ekleme:** n elemanlı tek yönlü listede sonu bulmak için bağlantılar üzerinde ilerleyerek n adım gitmek gerekir. Karmaşıklığı $O(n)$'dir. Sona ekleme, gelen elemanın mutlaka en sonda olması gerektiğinde yapılır. Örneğin bir kuyrukta ilk gelen işe ilk hizmet edilir, o yüzden yeni gelen en sona eklenmelidir.
- **Başa ekleme:** Tek adımda yapılır, karmaşıklığı $O(1)$'dir.
- **Araya ekleme:** Listedeki elemanlara sıralı erişiliyorsa, yeni elemanı sırayı bozmayacak şekilde uygun yere yerleştirmek için yapılır.

> **Hocanın vurgusu:** Herhangi bir hiyerarşi, sıra ihtiyacı yoksa doğrusu başa eklemektir. Niye sona kadar gideyim? Başa eklemenin karmaşıklığı $O(1)$'dir ve zaman kazandırır.

3. Tek Yönlü Bağlı Listede Düğüm Oluşturma

Hoca, işlemler çok algoritmik olmadığı için akış diyagramı çizmeden doğrudan kod yazarak ilerledi; algoritmik bir konu olduğunda akış çizileceğini söyledi. Bellek ayırma fonksiyonları `stdlib.h` içinde olduğu için bu başlık dosyası eklenir.

```c
#include <stdio.h>
#include <stdlib.h>   /* malloc burada tanımlı */

typedef struct node {
    int value;
    struct node *next;
} node;

node *head;   /* bu derste kolaylık için global */
```

3.1 Global `head` Tercihi

Hoca `head`'i global tanımladı, çünkü hemen bütün fonksiyonlar (`createNode` hariç) ona ihtiyaç duyacaktı. `head` parametre olarak fonksiyondan fonksiyona aktarılarak da kullanılabilir; ikisi de mümkündür. Bu tamamen uygulamaya göre verilecek bir karardır.

> **Hocanın vurgusu:** Gereksiz yere global değişken tanımlamayın. Hata ayıklamada takip etmek çok zorlaşır.

3.2 `main` İçinde Kullanım

Hoca fonksiyonların nasıl kullanılacağını önce `main` üzerinde gösterdi. Başlangıçta liste boş olduğu için `head`'e `NULL` atanır. Sonra bir düğüm için yer açılır ve başa 8 eklenir; ardından yeni bir düğüm açılıp sona 7 eklenir.

```c
int main()
{
    node *ptr;
    head = NULL;              /* başlangıçta liste boş */
    ptr = createNode();
    insertAtFront(ptr, 8);
    ptr = createNode();
    insertAtEnd(ptr, 7);
    /* ... */
    return 0;
}
```

`createNode` ayırdığı yerin adresini döndürüp `ptr`'ye atadığı için dönüş tipi `node *` olmalıdır. Hoca, yazılan bütün fonksiyonların kullanılmadan önce bildirilmesi (prototype) gerektiğini, kendisinin derste bunları tek tek yazmayacağını ama öğrencilerin atlamaması gerektiğini söyledi.

Soru: Fonksiyon bildirimlerini `main`'in üstünde yapmıyor muyduk? **Cevap:** Hoca, bildirimin fonksiyonun kullanılacağı yerde yapılması gerektiğini söyledi. Fonksiyonlar `main` içinden çağrılacaksa bildirim `main` içinde, `insertAtFront` içinden çağrılacaksa onun içinde yapılır. Dışarıda (global olarak) bildirmenin anlamı, "bu fonksiyonları bütün fonksiyonların içinde kullanacağım" demektir. Gerçekten öyleyse sorun yoktur, değilse kafa karıştırıcıdır.

> **Ders sonrası not:** Hoca dışarıda bildirmenin "doğru bir şey olmadığını" söyledi. C'de iki yol da geçerlidir: Blok içindeki bildirim yalnız o blokta görünür, dosya başındaki bildirim dosyanın geri kalanında görünür. Bu notun derleme testinde `main` içinde yazılan `node *createNode();` bildirimi `gcc -Wall` ile uyarısız derlendi. Yaygın

uygulama ise prototipleri dosya başına ya da bir başlık (header) dosyasına yazmaktır; ödevlerde hocanın tercihine uymak gerekir.

3.3 `createNode`

```
node *createNode()
{
    node *ptr;
    ptr = (node *) malloc(sizeof(node));
    if (ptr == NULL)
        exit(1);          /* yer ayrılamadı */
    ptr->next = NULL;
    return ptr;
}
```

Yalnız bir düğüm için yer tutulacağı için `sizeof(node)` kadar yer istemek yeterlidir; `malloc`'un döndürdüğü adres `node *` tipine dönüştürülür (cast). `malloc` sonucu `NULL` ise yer tutulamamış demektir. Hoca burada `exit` ile programdan tamamen çıkmayı tercih etti, çünkü ders bir programlama dersi değildi. Gerçek uygulamada bu durumda ne yapılacağına algoritmaya göre karar verilir; programı terk etmek zorunlu değildir.

Yer tutulduysa yeni düğümün `next`'ine `NULL` atanır. Örneğin 9900 numaralı adreste bir düğüm ayrılmıştır; bu düğüm henüz kimseyi göstermez, "boşlukta duran" bir değişkendir. Fonksiyon bu yerin başlangıç adresini (9900) döndürür ve `main` bunu `insertAtFront`'a gönderir.

Soru (hocanın sorusu): Yer tutma ne zaman başarısız olur? **Cevap:** En temel sebep yer olmamasıdır. Hoca, yer tutulamadığı hâlde tutuldu sanılıp işlem yapılmaya kalkılmaması için bu kontrolün mutlaka yapılması gerektiğini söyledi.

> **Hocanın vurgusu:** `malloc`'tan sonra `NULL` kontrolünü mutlaka yapın. Yer tutulamamışsa o anda hiçbir şey olmamıştır; üzerinde işlem yapmaya kalkmayın.
>
> **Ders sonrası not:** Hoca yer tutulamama sebepleri arasında "read-only bir bellek bölgesiyle ilgili bir şey yapmak" da saydı. `malloc` istenen boyutta yer ayıramadığında `NULL` döndürür; salt okunur bir bölgeye yazmaya çalışmak ise `malloc`'un sonucu değil, çalışma sırasında oluşan ayrı bir hatadır (örneğin segmentation fault).
>
> **Ders sonrası not:** Hoca `createNode` içinde değişkeni bir yerde `p`, bir yerde `ptr` diye yazdı ve fark edip `ptr` olarak düzeltti. Değişken adı fonksiyon boyunca aynı olmalıdır.

4. Başa Ekleme

```
void insertAtFront(node *newNode, int value)
{
    newNode->value = value;
    newNode->next = head;
    head = newNode;
}
```

Fonksiyon, yeri önceden ayrılmış düğümün adresini ve yerleştirilecek değeri alır. Üç satırda iş biter: Değer yazılır, yeni düğüm eski ilk düğümü gösterir, `head` yeni düğümü gösterir.

4.1 Adım Adım İzleme

Hoca aynı fonksiyonla art arda üç ekleme yaptı:

- **8 ekleniyor (9900):** Başlangıçta `head = NULL`'dır. `9900->value = 8` yazılır. `9900->next = head` olduğu için `next`'e zaten `NULL` gelir. `head = 9900` olur. Liste: 8.
- **12 ekleniyor (8500):** `main`'de yeni bir yer ayrılmış, 8500 numaralı adres tutulmuştur. `8500->value = 12`. `8500->next = head`, yani 9900; yeni düğüm 8'in düğümünü gösterir. `head = 8500`. Liste: 12 → 8.
- **25 ekleniyor (12000):** `12000->value = 25`. `12000->next = head`, yani 8500. `head = 12000`; `head`'in 8500'e olan bağlantısı kopar. Liste: 25 → 12 → 8.

Adım	İşlem	Liste
Başlangıç	`head = NULL`	(boş)
8 eklenir (9900)	`9900->next = NULL`, `head = 9900`	8
12 eklenir (8500)	`8500->next = 9900`, `head = 8500`	12 → 8
25 eklenir (12000)	`12000->next = 8500`, `head = 12000`	25 → 12 → 8

Bu üç ekleme derlenip çalıştırıldığında listenin `25 12 8` olarak yazdırıldığı görüldü. Liste boşken ayrı bir kontrol gerekmez: `head` zaten `NULL` olduğu için yeni düğümün `next`'i `NULL` olur.

Hoca, verdiği adreslerin (8500, 9900, 12000) bu kadar dağınık olmadığını da açıkladı. Bellekte arka arkaya boş yer varsa düğümler genelde yakın yerlere yerleşir; arada dolu bir bölge varsa sonraki düğüm başka bir yere geçebilir. Bu "uçuk" adresleri, düğümlerin ardışık olduğu izlenimi oluşmasın diye seçmişti.

Soru: Üçüncü satırda `head = newNode` olmayacak mı? **Cevap:** Hoca öğrenciye hak verdi: Tahtadaki işaret yanlış yazılmıştı, doğrusu atamadır (`head = newNode;`).

Soru: `insertAtFront` yazılırken parametre adı ne yazılmıştı? **Cevap:** Fonksiyon içinde kullanılan ad `newNode`'dur; hoca parametre listesini buna göre düzeltti.

Başa eklemenin karmaşıklığı $O(1)$'dir. Gelen elemanın nereye eklendiğinin önemi yoksa başa eklemek zaman kazandırır.

5. Sona Ekleme

Sona ekleme, özel bir sebeple yeni gelenin mutlaka en sonda olması gerektiğinde yapılır; örneğin kuyruğa en son gelen elemanı kuyruğun sonuna eklemek istediğimizde. Tek yönlü listede yalnız ileri yönde gidebildiğimiz için sona eklemek, listenin sonunu bulmak demektir. Son düğüm, `next`'i `NULL` olan düğümdür.

Fonksiyon bir şey döndürmez (`void`), çünkü yalnız gelen bilgiyi yerleştirir. İki durum vardır:

- **Liste boşsa** (`head == NULL`): Henüz hiç düğüm yoktur. Yapılacak tek şey `head`'in yeni düğümü göstermesini sağlamaktır.
- **Liste doluysa:** Listenin sonu bulunur ve son düğümün `next`'ine yeni düğüm bağlanır.

```
void insertAtEnd(node *newNode, int value)
{
    node *temp;
    newNode->value = value;
    if (head == NULL)               /* liste boş */
        head = newNode;
    else {                          /* son düğümü bul */
        temp = head;
        while (temp->next != NULL)
            temp = temp->next;
        temp->next = newNode;
    }
}
```

5.1 Adım Adım İzleme

Liste: `head -> 1000 [7] -> 1200 [5] -> 1400 [9] -> NULL`. (Hoca son düğüme önce 7 yazdı, karışmasın diye 9 yaptı.) Yeni düğüm 1600 adresindedir, `next`'i `NULL`'dır ve değeri 27 olacaktır.

`temp`	`temp->next`	Karar
1000	1200	`NULL` değil, ilerle

temp	temp->next	Karar
1200	1400	NULL değil, ilerle
1400	NULL	Dur, 1400->next = 1600

Döngüden çıkıldığında temp son düğümü (1400) gösterir. temp->next = newNode ile 1400'ün next'i artık NULL değil 1600 olur. Liste: 7 → 5 → 9 → 27. Bu iz Python'la, sonuç da C programıyla (7 5 9 27) doğrulandı.

> **Ders sonrası not:** Hoca tahtada insertAtEnd'in parametre listesine önce int value'yu yazmayı unuttu, sonra ekledi. Kod bu nottaki hâliyle tamdır.

Soru: head == NULL diye kontrol ettik ama head'e hiçbir yerde atama yapmadık; head'i parametre olarak da almadık. **Cevap:** Hoca önce, elinde ekleye ekleye bu hâle gelmiş dolu bir liste olduğunu varsaydığını söyledi: Liste boş olsaydı head doğrudan yeni düğümü gösterecekti, doluysa sonu bulunup ekleme yapılacak. Parametre konusunda ise head'in derse başlarken global tanımlandığını hatırlattı; global olduğu için bütün fonksiyonlar ona erişebilir. İstenirse fonksiyondan fonksiyona parametre olarak da aktarılabilir.

Sona eklemenin karmaşıklığı $O(n)$'dir.

6. Sıralı Listeye Araya Ekleme (Alıştırma)

Moladan sonra hoca araya eklemeyi öğrencilerin kendisinin yazmasını istedi. Uzaktan eğitimde dersin akılda kalma ihtimalinin düşük olduğunu, kalıcılığı sağlamanın en etkili yolunun derste anlatılanların kodunu oturup yazmak olduğunu söyledi. Algoritmanın yolunu verdi, kodu vermedi.

Liste küçükten büyüğe sıralıdır: 1000'de 5, 1200'de 7, 1300'de 11. Yeni düğüm 1400 numaralı adrestedir, next'i NULL'dır ve değeri 8'dir. Sıra bozulmadan yerleştirilmesi istenir.

- 8, 5 ile karşılaştırılır: 8 büyük, ilerlenir.
- 1200'e gelinir, 7 ile karşılaştırılır: 8 büyük, ilerlenir.
- 1300'e gelinir, 11 ile karşılaştırılır: 8 küçük. 8'in 7 ile 11 arasına girmesi gerektiği ancak burada anlaşılır.
- 7'nin düğümü (1200) artık 1400'ü, 1400 de 1300'ü göstermelidir.

Sorun şudur: 8'i 11 ile karşılaştırdığımız anda 1200 adresini kaybetmişizdir. 7'nin düğümünün next'ini değiştirebilmek için 11'e nereden gelindiğini, yani bir önceki düğümün adresini ayrı bir değişkende saklamak gerekir. Yoksa yerleştirme yapılamaz. Hoca, sonraki konularda gösterilecek benzer örneklerden (silme) de faydalanılabileceğini söyledi. Çözüm kodu Örnek Sorular bölümündedir.

> **Hocanın vurgusu:** Araya eklemede ve silmede bir önceki düğümün adresini kaybetmeyin; onu yedek bir değişkende tutun.

7. Listeyi Ekrana Yazdırma

Listedeki elemanlar, listede bulunma sırasına göre ekrana yazdırılır. Liste üzerinde ilerlemek için bir node * değişkeni kullanılır ve listenin başından (head) başlatılır. Bu, adım sayısı belli olmayan bir döngüdür: Bakılan adres NULL'dan farklı olduğu sürece o adresteki değer yazdırılır ve bir sonraki adrese geçilir.

```
void display()
{
    node *ptr = head;
    while (ptr != NULL) {
        printf("%d ", ptr->value);
        ptr = ptr->next;
    }
}
```

Örnek: `head -> 1000 [5] -> 1200 [9] -> NULL`.

`ptr`	`ptr != NULL`?	Ekrana	Sonraki `ptr`
1000	Evet	5	1200
1200	Evet	9	`NULL`
`NULL`	Hayır, döngüden çık		

Ekrana `5 9` yazılır.

8. Listeden Eleman Silme

Silmede dışarıdan bir değer verilir ve o elemanın listeden çıkarılması istenir.

> **Hocanın vurgusu:** Problem çözerken önce kâğıt üzerinde bir örnek çizip onun üzerinde çalışın. Böylece olası durumları (case) rahat görürsünüz, algoritmayı tasarlamak kolaylaşır ve problemi daha hatasız çözersiniz.

8.1 Olası Durumlar

Hoca önce `head -> 1000 [8] -> 1200 [9] -> 1300 [7] -> NULL` listesini çizip durumları tartıştı:

- **Liste boş** (`head == NULL`): Aranan eleman zaten yoktur, silinecek bir şey de yoktur. İlk kontrol budur; boşsa hemen çıkılır. Hoca, programı kullananların gereksiz hatalara yol açacak işler yapabileceğini, bu yüzden bu kontrolün dikkatle yapılması gerektiğini söyledi.
- **Silinecek eleman ilk eleman:** Örneğin 8 silinecekse, `head` ayrı bir değişken olduğu için değiştirilmelidir. `head` artık 1000'i değil 1200'ü göstermelidir.
- **Silinecek eleman herhangi bir yerde:** En kolay durumdur. Örneğin 9 silinecekse 1000'de 9 yoktur, 1200'de vardır. 8'in düğümü artık 1200'ü değil 1300'ü gösterir. 9'a kimse erişemediği için 9 listeden çıkmış olur. Bunun için de 1200'ü kimin gösterdiğini, yani bir önceki adresi bilmek gerekir; mantık araya eklemedeki ile aynıdır.
- **Eleman listede yok:** Algoritma yazılırken ortaya çıkan dördüncü durumdur; liste sonuna kadar gidilir ve "not found" verilir.

8.2 Silme Fonksiyonu

Hoca algoritmayı `head -> 1000 [7] -> 1200 [5] -> 1300 [11] -> NULL` listesi üzerinde yazdı. Silinecek değer `del` değişkenindedir.

```
void deleteNode(int del)
{
    node *temp, *prev, *current;
    if (head == NULL)
        return;                              /* liste boş */
    if (head->value == del) {                /* ilk eleman */
        temp = head;
        head = head->next;
        free(temp);
    }
    else {
        prev = head;
        current = head->next;
        while (current != NULL && current->value != del) {
            prev = current;
            current = current->next;
        }
        if (current == NULL)
            printf("not found");
```

```
        else {
            prev->next = current->next;
            free(current);
        }
    }
}
```

Döngünün iki koşulu vardır. `current != NULL` koşulu "eleman listede yok" durumunu yakalar: `current` `NULL` olduysa bakılacak eleman kalmamıştır. `current->value != del` koşulu ise "henüz bulunamadı" demektir. İkisi de doğru olduğu sürece döngü döner.

Döngüden çıkmanın iki sebebi olabilir: Ya liste bitmiştir ya da silinecek eleman bulunmuştur. Kullanıcıya anlamlı bir cevap verebilmek için döngüden sonra hangisinin olduğu kontrol edilir: `current == NULL` ise "not found" yazılır; değilse önceki düğüm, silinen düğümün sonrasını gösterir ve silinen düğüm `free` ile serbest bırakılır.

8.3 Adım Adım İzleme

7 siliniyor (ilk eleman): `head->value == 7` olduğu için `temp = head = 1000` tutulur. `head = head->next = 1200`; `head` artık 5'in düğümünü gösterir. 1000 numaralı düğüm hâlâ 1200'ü gösterir, ondan kurtulmak gerekir: `free(temp)` ile 1000 için tutulan yer serbest bırakılır. 1000 numaralı adres artık başkaları tarafından kullanılabilir.

11 siliniyor (aradaki/sondaki eleman): Hoca bu izi, 7'nin silinmediğini varsayarak `prev = head = 1000`, `current = head->next = 1200` ile başlattı.

prev	current	current->value	Karar
1000	1200	5	11 değil, `prev = 1200`, `current = 1300`
1200	1300	11	Bulundu, döngüden çık

`current` `NULL` değildir. `prev->next = current->next` ile 1200'ün `next`'i 1300'ün `next`'i olur, yani `NULL`. `free(current)` ile 1300 serbest bırakılır. Liste: 7 → 5. Bu iz Python'la doğrulandı.

Liste yalnız `[5]` iken 8 silinmeye çalışılıyor: Önceki iki silmeden sonra listede yalnız 1200'deki 5 kalmıştır. `head == NULL` mı? Hayır, `head = 1200`. `head->value == 8` mi? Hayır, 5. `prev = 1200`, `current = 1200->next = NULL`. Döngü koşulu ilk kontrolde yanlıştır, döngüye girilmez. `current == NULL` olduğu için "not found" yazılır.

Böylece dört durum da kontrol edilmiş olur: liste boş, ilk eleman, herhangi bir yerdeki eleman, listede olmayan eleman. C programında 7, 11, 8 ve 5 sırasıyla silindiğinde liste `5` `11`, `5`, "not found" ve boş liste olarak beklenen sonuçları verdi; boş listede silme çağrısı hatasız döndü.

> **Ders sonrası not:** Hoca durumları tartışırken "kimse 9'u göstermeyecek, `free`'yi bırakacağız, o ayrı konu" dedi. Bağlantıyı koparmak elemanı listeden çıkarır, ama yazılan kodda olduğu gibi `free` de çağrılmalıdır; çağrılmazsa o düğümün belleği program sonuna kadar boşa tutulur (bellek sızıntısı, memory leak).

9. Çift Yönlü Bağlı Liste

Tek yönlü listede silme ve araya eklemede bir önceki adresi ayrı bir değişkende tutmak gerekiyordu. Örneğin 7'yi silmeye karar verdiğimizde `head`'in 7'den sonrakini, 5'i silmeye karar verdiğimizde 7'nin 5'ten sonrakini göstermesi gerekir; bunun için 5'e nereden gelindiğini bilmek şarttır. Bağlı listenin kullanıldığı pek çok yerde önceki elemanın yerini bilmek gerektiği için çift yönlü bağlı liste (doubly linked list) tasarlanmıştır: Her düğüm hem sonrakini hem öncekini gösterir.

Hocanın örneği: 1000'de 9, 1200'de 7, 1300'de 8. İleri yönde 1000 → 1200 → 1300 → `NULL`. Geri yönde 1300'ün öncesi 1200, 1200'ün öncesi 1000'dir. 1000'den önce kimse olmadığı için 1000'in öncesi `NULL`'dır. Listenin başına front, sonuna end denir.

```
NULL <- [prev | 9 | next] <-> [prev | 7 | next] <-> [prev | 8 | next] -> NULL
             (1000)               (1200)               (1300)
```

Düğüm yapısına `prev` alanı eklenir; her düğüm `prev`, `value`, `next` alanlarından oluşur.

```
typedef struct node {
    struct node *prev;
    int value;
    struct node *next;
} node;

node *head;   /* yine global; uygulamaya göre parametre olarak da aktarılabilir */

node *createNode()
{
    node *ptr = (node *) malloc(sizeof(node));
    if (ptr == NULL)
        exit(1);
    ptr->next = NULL;
    ptr->prev = NULL;
    return ptr;
}
```

`createNode` artık `prev`'e de `NULL` atar. Örneğin 2000 numaralı adreste tutulan düğümün öncesi de sonrası da kimseyi göstermez.

9.1 Başa Ekleme

Örnek: `head -> 1500 [5] <-> 1800 [2]`. 1800'ün `prev`'i 1500, 1500'ün `prev`'i `NULL`'dır. Yeni düğüm 1700 adresindedir, değeri 12'dir, `prev` ve `next`'i `NULL`'dır. 1700'ün yeni front olması için üç şey gerekir: `head` 1700'ü, 1700 1500'ü, 1500'ün `prev`'i 1700'ü göstermelidir.

```
void insertAtFront(node *newNode, int value)
{
    newNode->value = value;
    if (head == NULL)               /* liste boş */
        head = newNode;
    else {
        head->prev = newNode;
        newNode->next = head;
        head = newNode;
    }
}
```

- `head->prev = newNode`: `head` 1500'dür, 1500'ün `prev`'i `NULL`'dan 1700'e döner.
- `newNode->next = head`: 1700'ün `next`'i 1500 olur.
- `head = newNode`: `head` 1700 olur; `head`'in 1500'e olan bağlantısı kopar.

Sonuç: `head -> 1700 [12] <-> 1500 [5] <-> 1800 [2]`. 1700'ün `prev`'i `NULL` kalır. 1500 hem öncesindeki 1700'ü hem sonrasındaki 1800'ü gösterir. C programında bu liste ileri yönde `12 5 2`, geri yönde `2 5 12` olarak yazdırıldı.

9.2 Sona Ekleme

> **Hocanın vurgusu:** Her zaman çizip bakın.

Örnek: `head -> 1500 [9] <-> 1700 [8]`. 1500'ün öncesi `NULL`, 1700'ün öncesi 1500, sonrası `NULL`'dır. Yeni düğüm 1900 adresindedir. Liste boşsa yine `head = newNode` yeterlidir. Değilse tek yönlü listedeki gibi liste boyunca ilerlenip son düğüm bulunur.

```
void insertAtEnd(node *newNode, int value)
{
    node *temp;
    newNode->value = value;
    if (head == NULL)
        head = newNode;
```

```
    else {
        temp = head;
        while (temp->next != NULL)
            temp = temp->next;
        temp->next = newNode;
        newNode->prev = temp;
    }
}
```

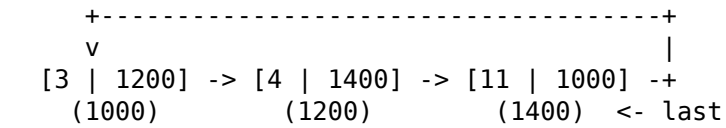
- `temp = 1500`; 1500'ün `next`'i 1700, `NULL` değil, `temp = 1700`.
- 1700'ün `next`'i `NULL`, döngüden çıkılır.
- `temp->next = newNode`: 1700'ün `next`'i 1900 olur.
- `newNode->prev = temp`: 1900'ün `prev`'i 1700 olur; yeni düğüm geriye doğru da bağlanır.

Tek yönlü listeden tek farkı, yeni düğümün `prev`'inin de bağlanmasıdır. Hoca burada şunu vurguladı: Sürekli ayrı bir `previous` değişkeninde nereden gelindiğini saklamakla uğraşmak yerine, düğümün kendi `prev` alanı öncekine erişimi sağlar. (Hoca değer atamasını tahtada en sona yazdı; yerinin önemi yoktur.)

10. Dairesel Bağlı Liste

Dairesel bağlı listede (circular linked list) son eleman `NULL`'ı değil ilk elemanı gösterir. Hoca bunun özellikle kuyruk yapısında çok kullanılan bir mantık olduğunu, kuyruklar anlatılırken yeniden bahsedileceğini söyledi. Faydası, bazı ihtiyaçlarda listeye dairesel bir şekilde erişebilmektir.

Örnek: 1000'de 3, 1200'de 4, 1400'de 11. 3 → 4 → 11'dir; 11 normalde son eleman olduğu için `NULL`'ı gösterirdi, dairesel listede 1000'i gösterir.



Bu kez baş değil son düğümün yeri tutulabilir. Hoca bunun birçok yoldan biri olduğunu belirtti. `last` 1400'ü gösterir; 1400 de 1000'i, yani başı gösterdiği için başa `last->next` üzerinden erişilir. Düğüm yapısı değişmez: Dairesel olmasına rağmen yapı tek yönlü listeninkiyle aynıdır, bir `value` ve bir `next` vardır. Başlangıçta liste boştur, `last = NULL`'dır.

10.1 Sona Ekleme

Hoca bu kodu başa ekleme olarak yazmaya başladı; `last = newNode` satırını yazınca bunun aslında sona ekleme olduğunu fark etti ve öyle adlandırdı.

```
void insertAtEnd(node *newNode, int value)
{
    newNode->value = value;
    if (last == NULL) {             /* liste boş */
        last = newNode;
        newNode->next = last;       /* kendini gösterir */
    }
    else {
        newNode->next = last->next; /* yeni düğüm ilk düğümü gösterir */
        last->next = newNode;       /* eski son düğüm yeni düğümü gösterir */
        last = newNode;             /* yeni düğüm artık son düğüm */
    }
}
```

Liste boşsa: Başka çare yoktur, `last` yeni düğümü gösterir. Dairesellik için yeni düğümün `next`'i `last`'ı, yani kendisini gösterir. Hocanın örneğinde 1200'deki 9 eklendiğinde `last = 1200` ve `1200->next = 1200` olur. Başka eleman olmadığı için düğüm kendini gösterir.

Liste doluysa: Örnek liste 3 → 4 → 11 → (3), `last = 1400`. Yeni düğüm 1700 adresindedir, değeri 15'tir.

- `newNode->next = last->next`: 1400'ün `next`'i 1000'dir; 1700'ün `next`'i 1000 olur. Başa dönebilmek için gerekir.
- `last->next = newNode`: 1400'ün `next`'i 1000 değil 1700 olur; 1400'ün 1000'e olan bağlantısı kopar.
- `last = newNode`: `last` 1400 değil 1700 olur.

Sonuç: 3 → 4 → 11 → 15 → (3). En son eleman 1700'dür ve başı (1000) gösterir. Bu sonuç Python izi ve C programıyla (`3 4 11 15`) doğrulandı.

> **Ders sonrası not:** Hoca izi bitirirken önce "gelen elemanı başa eklemiş gibi oldum" dedi, hemen ardından yaptığının sona ekleme olduğunu fark edip düzeltti. Doğrusu: Bu kod 15'i listenin sonuna ekler.

10.2 Başa Ekleme

Boş liste kısmı sona eklemedekiyle aynıdır. Fark yalnız `else` kısmındadır.

Örnek: 1000'de 9, 1200'de 8; 9 → 8 → (9), `last = 1200`. Yeni düğüm 1300 adresindedir, değeri 12'dir. 12'yi başa almak demek, 8'in düğümünün artık 1000'i değil 1300'ü göstermesi, 1300'ün de 1000'i göstermesi demektir. Son eleman 1300'ü gösterirse 12 listenin başı olur. `last` değişmez, hâlâ 1200'dür.

```
void insertAtFront(node *newNode, int value)
{
    newNode->value = value;
    if (last == NULL) {
        last = newNode;
        newNode->next = last;
    }
    else {
        newNode->next = last->next;
        last->next = newNode;       /* last değişmez */
    }
}
```

- `newNode->next = last->next`: `last` 1200'dür, 1200'ün `next`'i 1000'dir; 1300'ün `next`'i 1000 olur.
- `last->next = newNode`: 1200'ün `next`'i 1000 değil 1300 olur.

Sonuç: 12 → 9 → 8 → (12), `last = 1200`. C programında liste `12 9 8` olarak yazdırıldı ve `last->value` 8 çıktı.

Başa ve sona ekleme aynı iki bağlantı işlemini yapar. Tek fark, sona eklemede `last`'ın yeni düğüme taşınmasıdır.

> **Ders sonrası not:** Hoca bu kodu bitirince "elemanı dizinin sonuna yerleştirmiş oluyoruz" dedi. Bu bir dil sürçmesidir: `last` değişmediği için eleman listenin başına eklenir. Ayrıca altyazıda "12'de artık 8000'i göstermediği için 12000'i gösterecek" diye geçen cümle, çizime göre "1300, 1000'i gösterecek" olmalıdır.

10.3 Dairesel Listeyi Yazdırma

Örnek liste: 300'de 11, 500'de 7, 900'de 8; 300 → 500 → 900 → (300), `last = 900`. Yazdırmada da önce listenin boş olup olmadığı kontrol edilir: `last == NULL` ise liste boştur. Değilse ilk eleman `last->next`'tedir (900'ün `next`'i 300, değeri 11). Yazdırma oradan başlar ve biri tekrar 300'ü gösterdiğinde, yani başa dönüldüğünde biter.

Hoca önce şu döngüyü yazdı:

```
for (ptr = last->next; ptr != last->next; ptr = ptr->next)
    printf("%d ", ptr->value);
```

Soru: Böyle yazarsak döngüye girilmez, değil mi? **Cevap:** Hoca önce izi yürütmeye başladı. Öğrenci itirazını açıkladı: `ptr last->next` ile başlatılıyor ve ilk kontrol `ptr != last->next` olduğu için koşul hemen yanlış oluyor. Hoca, karşılaştırma için `for (i = 0; i < 0; i++)`

döngüsünün ilk kontrolde koşul yanlış olduğundan hiç çalışmadığını hatırlattı ve öğrencinin haklı olduğunu kabul etti. Derleme testinde bu `for` döngüsü 11 → 7 → 8 listesinde 0 eleman yazdırdı.

Hoca bir ara `ptr`'yi `last`'tan başlatmayı denedi; bir öğrenci bu durumda bütün elemanların yazılmayacağını söyledi.

Hoca sonunda `ptr`'nin `last->next`'ten başlaması gerektiğini, ama kontrolün döngünün sonunda yapılması gerektiğini söyleyip `do-while` ile düzeltti:

```
void display()
{
    node *ptr;
    if (last == NULL)
        printf("list is empty");
    else {
        ptr = last->next;            /* ilk düğüm */
        do {
            printf("%d ", ptr->value);
            ptr = ptr->next;
        } while (ptr != last->next);
    }
}
```

`ptr`	Yazılan	Sonraki `ptr`	`ptr != 300`?
300	11	500	Evet, devam
500	7	900	Evet, devam
900	8	300	Hayır, döngüden çık

Ekrana `11 7 8` yazılır. Bu iz Python'la, çıktı C programıyla doğrulandı; boş listede program "list is empty" yazdı.

> **Ek bilgi (derste anlatılmadı):** `while` ve `for` döngüleri koşulu gövdeden önce, `do-while` ise gövdeden sonra kontrol eder; bu yüzden `do-while` gövdesi en az bir kez çalışır. Dairesel listede başlangıç düğümü aynı zamanda bitiş işareti olduğu için, ilk düğümü yazdırmadan önce kontrol yapan bir döngü hemen durur. `do-while` bu sorunu ortadan kaldırır.

Örnek Sorular

Bu derste hazır kare ve video yoktur; sorular derste sözlü sorulan sorulardan alınmıştır.

Soru 1 (derste sözlü, 00:12:14): `malloc` ile yer tutma ne zaman başarısız olur, bu durumda ne yapılmalıdır?

> **Çözüm:** En temel sebep bellekte yer olmamasıdır. Bu durumda `malloc` `NULL` döndürür. Sonuç mutlaka `NULL` ile karşılaştırılmalı, yer tutulmadıysa düğüm üzerinde işlem yapılmamalıdır. Hoca bu durumda `exit(1)` ile çıktı; uygulamada başka bir davranış da seçilebilir.

Soru 2 (derste sözlü, 00:16:52): n elemanlı tek yönlü bağlı listede yeni bir elemanı en sona eklemenin karmaşıklığı nedir?

> **Çözüm:** Derste bir öğrenci $O(n)$ diye cevap verdi, hoca onayladı. Listenin sonuna ulaşmanın başka yolu yoktur; bağlantılar üzerinde ilerleyerek n adım gidilir.

Soru 3 (derste sözlü, 00:17:30): Yeni gelen bir elemanı ne zaman sona eklemek isteriz?

> **Çözüm:** Sıranın önemli olduğu durumlarda; örneğin bir kuyrukta ilk gelen işe ilk hizmet edilir, bu yüzden yeni gelen en sona eklenir. Bir hiyerarşi yoksa doğrusu başa eklemektir.

Soru 4 (derste sözlü, 00:35:05): Başa eklemenin (`insertAtFront`) karmaşıklığı nedir?

Çözüm: $O(1)$. Liste ne kadar uzun olursa olsun yalnız üç atama yapılır: değer yazılır, `newNode->next = head`, `head = newNode`.

Soru 5 (derste sözlü, 01:00:26): Küçükten büyüğe sıralı `5 → 7 → 11` listesine (adresler 1000, 1200, 1300), 1400 adresindeki 8 değerli düğümü sırayı bozmadan yerleştiren algoritmayı yazın.

Çözüm (derste verilmedi): Hoca yalnız yolu verdi: Karşılaştırarak ilerlenir, önceki düğümün adresi ayrı bir değişkende tutulur. Aşağıdaki fonksiyon bu yolu izler. Yeni değer ilk elemandan küçükse ya da liste boşsa `head` değişir. Aksi hâlde `prev`/`current` ile, değeri yeni değerden küçük olan düğümler geçilir; `current` bulunduğunda yeni düğüm `prev` ile `current` arasına bağlanır. Kod `gcc -Wall` ile uyarısız derlendi. 5, 7, 11, 8 sırasıyla eklendiğinde liste `5 7 8 11`; ardından 2 ve 20 eklendiğinde `2 5 7 8 11 20` oldu. Örnekte 8 için döngü 7'de ilerler, 11'de durur: `prev = 1200`, `current = 1300`; `1400->next = 1300`, `1200->next = 1400`.

```c
void insertSorted(node *newNode, int value)
{
    node *prev, *current;
    newNode->value = value;
    if (head == NULL || value < head->value) {   /* boş liste ya da en küçük */
        newNode->next = head;
        head = newNode;
    }
    else {
        prev = head;
        current = head->next;
        while (current != NULL && current->value < value) {
            prev = current;
            current = current->next;
        }
        newNode->next = current;    /* sona eklenirse current == NULL */
        prev->next = newNode;
    }
}
```

Soru 6 (derste sözlü, 02:24:50): Dairesel listeyi yazdırmak için yazılan `for (ptr = last->next; ptr != last->next; ptr = ptr->next)` döngüsüne girilir mi?

Çözüm: Girilmez. `ptr` `last->next` ile başlatılır ve ilk kontrol `ptr != last->next` hemen yanlış olur. Hocanın karşılaştırması olan `for (i = 0; i < 0; i++)` de aynı sebeple hiç çalışmaz. Çözüm, kontrolü gövdeden sonra yapan `do-while` döngüsüdür. Derleme testinde `for` sürümü 0 eleman, `do-while` sürümü `11 7 8` yazdırdı.

Dersin Toparlaması

Bu derste, eleman sayısı çalışma sırasında sürekli değişen veriler için bağlı listenin neden uygun olduğu görüldü ve tek yönlü bağlı listenin temel işlemleri yazıldı: düğüm oluşturma, $O(1)$ başa ekleme, $O(n)$ sona ekleme, ekrana yazdırma ve dört durumu (boş liste, ilk eleman, aradaki eleman, olmayan eleman) ele alan silme. Araya ekleme ve silmede önceki düğümün adresini tutma ihtiyacı, çift yönlü bağlı listeye geçişin gerekçesi oldu: Her düğümün `prev` alanı önceki düğüme doğrudan erişim sağlar. Son olarak, son düğümün başı gösterdiği dairesel bağlı listede `last` pointer'ı ile başa ve sona eklemenin aynı iki bağlantı işlemiyle yapıldığı, farkın yalnız `last`'ın güncellenmesinde olduğu, yazdırmada ise `do-while` gerektiği gösterildi.

Hoca bağlı listelerin bu dersle bittiğini söyledi. Bağlı listeler sonraki haftalarda kuyruk, ağaç ve graf yapılarında sürekli kullanılacak ve orada bilindiği varsayılacak. Bu yüzden başa, sona ve araya ekleme ile silme kodlarını derstekine bakmadan, gerekirse farklı bir biçimde kendi başınıza yazmanızı önerdi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.